

从代码瘦身到代码审美

可验证重构与解耦的工程学习论文

真正好的代码，不是看起来最短，而是让意图更可见、责任更有界、依赖更诚实，并且让每一次改变都能被验证、定位和撤回。

适用对象	有 C#/.NET 基础，希望提升代码判断力的开发者
学习方式	概念 - 案例 - 评审 - 练习 - 答案 - 迁移
核心标准	行为保持、边界诚实、风险可证、变更可逆
版本日期	2026-07-29

本文是自包含学习材料。生成文档不代表学习完成；请以练习、复述、真实代码迁移和复盘证据判断能力是否增长。

目录

目录条目与正文标题均写入 PDF 书签，可直接跳转。

摘要	6
核心论点	6
学习目标	6
如何学习本文	6
第一部分 从“短”到“可判断”的代码审美	8
1. 把代码审美变成可观察的工程语言	8
1.1 八个维度	8
1.2 为什么短代码不自动等于美	8
1.3 审美判断的最小单位	9
2. 信息隐藏：模块不是文件夹，而是变化的防火门	9
2.1 Parnas 的问题意识	9
2.2 按步骤切分不等于模块化	9
2.3 从变化轴反推边界	10
2.4 判断是否是真模块的四个实验	10
3. 耦合、内聚与依赖方向	10
3.1 不要把耦合只理解成“引用数量”	10
3.2 内聚不是“东西放在一起”	11
3.3 依赖反转、端口与适配器	11
3.4 稳定依赖方向不是“接口越多越好”	11
3.5 让边界可执行	12
4. 重复的判断：相同文字不一定是相同行为	12
4.1 四种“相同”	12
4.2 有意重复是债务，还是边界成本	12
4.3 为什么布尔开关万能函数常常更差	13
4.4 去重决策记录	13
5. 删除死代码与兼容层：静态沉默不等于运行时不存在	13
5.1 静态候选为什么不够	13
5.2 九类隐形入口	13
5.3 删除证据阶梯	14
5.4 兼容代码的四种状态	14
5.5 低频并不等于无价值	14
6. 行为保持重构：把“没有改功能”写成可检验命题	15
6.1 什么叫行为	15

6.2 基线不是“测试都绿”一句话	15
6.3 小而可逆的批次	15
6.4 测试层与风险相配	16
6.5 回滚是设计输入	16
6.6 观察行为差异	16
7. 指标：测量地图，不要奖励绕路	17
7.1 正确理解五类指标	17
7.2 一个紧凑的瘦身记分卡	17
7.3 防止指标游戏	17

第二部分 八个 C# 判断案例 19

8. 案例一：发票金额 - 把政策从流程噪声中显露出来	19
8.1 改前	19
8.2 推理	19
8.3 改后	19
8.4 安全证据	20
9. 案例二：库存保留 - 美感不能以并发正确性为代价	21
9.1 改前	21
9.2 推理	21
9.3 改后	21
9.3.1 适配器关键形状	23
9.4 安全证据	23
10. 案例三：通知客户端 - 用端口隔离副作用，不泄漏供应商语言	23
10.1 改前	23
10.2 推理	24
10.3 改后	24
10.4 安全证据	25
11. 案例四：导入校验 - 保留有意重复，拒绝布尔万能入口	26
11.1 改前的“去重”提案	26
11.2 推理	26
11.3 改后：只复用稳定的小核	26
11.4 安全证据	27
12. 案例五：报表查询 - 让数据粒度、空值和排序进入契约	27
12.1 改前	27
12.2 推理	28
12.3 改后	28
12.4 安全证据	29
13. 案例六：调度任务 - 零引用类型可能是生产入口	29
13.1 删除候选	29
13.2 推理与安全迁移	30

13.3 安全证据	30
14. 案例七：序列化兼容 - 内部重命名不应偷改线缆协议	30
14.1 改前	30
14.2 改后：把线缆契约与领域名称分开	31
14.3 安全证据	31
15. 案例八：纯运费规则 - 取消没有收益的 DI 仪式	31
15.1 改前	31
15.2 推理	32
15.3 改后	32
15.4 安全证据	32
16. 从案例提炼出的共同模式	33
第三部分 把审美带进代码评审	34
17. 按风险顺序阅读，而不是按个人偏好挑刺	34
17.1 第一问：这次到底要保持和改变什么	34
17.2 第二问：高损失边界在哪里	34
17.3 第三问：责任和依赖是否更诚实	34
17.4 第四问：重复真的属于同一知识吗	34
17.5 第五问：证据覆盖了真实失败机制吗	35
17.6 第六问：失败后能否定位和恢复	35
17.7 最后才问表达细节	35
18. 审美红旗：看到后应停下来问证据	35
19. 写出能推动改进的评审意见	36
可打印的一页代码审美评审卡	37
A. 先定性	37
B. 再找风险	37
C. 看边界	37
D. 判重复	37
E. 看证据	37
F. 准备恢复	37
一句话结论	37
第四部分 练习：把概念变成判断	38
练习说明	38
练习 1：诊断一个短函数	38
练习 2：比较两个通知设计	38
练习 3：重构草图 - 拆开导入编排	38
练习 4：边界设计 - 报表查询	38

练习 5：死代码证据	39
练习 6：库存并发	39
练习 7：是否去重	39
练习 8：改写评审意见	39
练习 9：指标判断	39
练习 10：设计一个可逆批次	39
 练习参考答案与解析	40
练习 1 解析	40
练习 2 解析	40
练习 3 解析	40
练习 4 解析	40
练习 5 解析	41
练习 6 解析	41
练习 7 解析	41
练习 8 解析	42
练习 9 解析	42
练习 10 解析	42
 第五部分 自测：能否把审美变成可验证判断	43
20. 十题自测	43
 自测答案、评分与补救路径	44
21. 参考答案	44
22. 评分与补救	44
 第六部分 反思与 14 天能力微练习	45
23. 一次真实评审后的反思模板	45
24. 14 天微练习路径	45
 参考文献与注释	46

摘要

代码审美不是对缩进、命名或行数的个人偏好，而是对一个系统能否被可靠理解、改变和验证的工程判断。ISO/IEC 25010:2023 的公开元数据将产品质量模型用于质量需求、评价、测试目标、验收与度量，但公开页面没有给出任何“代码必须减少多少行”的结论。[S1] .NET 的 CA1505 也把代码行数放在程序体积与圈复杂度共同构成的可维护性信号里，且阈值可配置；它不是删代码的授权。[S5]

本文提出一个可操作的核心命题：美的代码让正确意图可见，让变化被边界吸收，让风险能够被证据约束。因此，代码瘦身的第一目标不是变短，而是在行为、契约、数据、事务和外部副作用可验证不变的前提下，减少认知负担、不诚实依赖、重复决策和无证据兼容负担。Parnas 的原始研究将模块化与隐藏可能变化的设计决策联系起来，而不是把模块化等同于按执行步骤拆文件。[S2] Fowler 则把重构描述为一系列小的、保持行为的转换；小步让错误更易定位，也让系统持续处于可工作状态。[S12]

本文从八个可观察维度定义代码审美，讲解信息隐藏、内聚与耦合、依赖反转、端口与适配器、重复判断、动态入口下的死代码删除、行为保持重构和指标使用。八个 C# 业务案例展示“何时抽象、何时不抽象、用什么证据证明安全”。最后给出顺序化评审问题、可打印评审卡、练习与解析、自测、反思模板及 14 天微练习。

核心论点

好代码不是最少的代码，而是让下一次正确修改所需的事实更少、距离更近，同时让错误修改更容易被测试、边界和运行证据发现。

本文综合建议：可把“审美”拆成三个层次，而不是凭直觉说“优雅”。

1. 阅读层：意图、业务词汇、输入输出和失败方式是否可见。
2. 结构层：责任、变化和依赖是否落在诚实的边界内。
3. 证据层：行为是否可测试、变更是否可回退、运行是否可观察。

如果一段代码在阅读层很漂亮，却把事务边界藏掉，它仍然不美；如果一个架构图层次整齐，却让领域层引用数据库 SDK，它仍然不美；如果一次重构通过了单元测试，却改变了 JSON 字段、行锁语义或消息发送次数，它也没有被证明是安全的。以上三项是本文综合建议，用于训练审美判断，不是外部标准。

学习目标

完成本文的阅读和练习后，读者应能尝试做到以下事情。这里的“能”是学习目标，不代表任何读者已经完成或掌握。

1. 用八个可观察维度解释“这段代码为什么更好”，而不只说“更简洁”。
2. 识别按文件或层拆分但没有隐藏变化的伪模块化。
3. 判断一个接口是在修正依赖方向，还是只增加 DI 仪式。
4. 区分文字重复、行为重复、知识重复和变化方向不同的有意重复。
5. 为死代码或兼容层候选建立静态、动态、契约和回退证据链。
6. 设计行为保持的小批次重构，并写出事务、数据和外部副作用不变项。
7. 用 LOC、复杂度、耦合、重复与覆盖率排序风险，而不被数字支配。
8. 写出可执行、能帮助作者改进且不把偏好伪装成缺陷的评审意见。

如何学习本文

本文综合建议：不要一次读完后只留下概念印象。可采用三遍法。

- 第一遍只读摘要、每章的“判断句”和评审卡，建立总体地图。
- 第二遍亲手遮住“改后”代码，先写自己的方案，再比较案例中的权衡。
- 第三遍完成练习和自测，把低分项映射到 14 天微练习，而不是回头泛读全文。

阅读案例时，固定问四句话：气味是什么？真正的变化轴是什么？哪条依赖 方向需要保持？什么证据足以证明安全？这四问是本文综合建议，也是从“看起来更顺眼”走向“能解释并验证”的最短训练循环。

第一部分 从“短”到“可判断”的代码审美

1. 把代码审美变成可观察的工程语言

“漂亮”如果不能被拆解，就很容易成为资历压制：评审者说“不优雅”，作者 却不知道改什么。本文不取消个人品味，而是先建立一组可以讨论、验证和 反驳的维度。

1.1 八个维度

维度	可观察问题	反面信号	证据线索
意图可见	名称和流程是否回答“为何”	魔法值、含混动词	示例、断言、评审复述
责任有界	变化能否停在一个拥有者内	巨型服务、跨域修改	变更历史、依赖图
依赖诚实	策略是否依赖细节	领域引用 SDK	项目引用、构造参数
不变项显式	何者绝不能变	隐式事务、静默吞错	类型、断言、约束
局部推理	理解修改需看多少地方	全局状态、远程副作用	调用链、状态拥有者
可测试	风险能否被隔离重现	时间/网络写死	测试层次、替身边界
可逆	失败能否快速撤回	大爆炸式切换	小提交、开关、回滚
可观察	生产差异能否被定位	无关联 ID、无结果量	日志、指标、追踪

本文综合建议：意图可见。 **Process**、**Handle**、**DoWork** 不是天然错误，但当它们遮住“计算可开票金额”“保留库存”或“发布发货通知”时，读者必须 进入实现才能知道目的。一个名称应尽量暴露业务动作，而不是只暴露技术动作。

本文综合建议：责任有界。“一个类只做一件事”常被误读为“一个类只有 一个方法”。更实用的问题是：哪类变化会让它改变？税率政策、数据库供应商、 消息协议和报表列布局若各自独立变化，就不应由同一个类共同拥有。

本文综合建议：依赖诚实。 如果一个业务规则只有拿到 **DbContext**、HTTP 客户端和日志器才能计算，签名就在谎报它的真实需要。诚实的签名会把纯计算 所需的数据与外部副作用分开，也会让基础设施细节依赖业务定义的端口。Microsoft 的 .NET 架构指导用依赖反转说明基础设施与核心解耦，并在 Clean Architecture 示例中让基础设施依赖应用核心。[S3]

本文综合建议：不变项显式。 金额舍入、库存不得为负、同一幂等键只产生 一次外部通知、事务内要么全成要么全败，这些都不应只存在于资深开发者记忆。 它们可以通过类型、数据库约束、条件更新、测试和清晰注释被显式表达。

本文综合建议：局部推理。 如果理解一个 12 行方法需要同时查看 9 个 单例、3 段配置和 2 个拦截器，它并不比一个 40 行的纯函数更简单。局部推理 关注“得出正确结论所需跨越的边界数量”，而不是屏幕上显示的行数。

本文综合建议：可测试、可逆与可观察。 这三个维度构成变更的安全网：测试在发布前区分预期与意外，可逆性控制修复时间，可观察性在真实运行中 揭示未被测试覆盖的差异。它们不能相互替代；日志不是测试，测试也不是生产 证明，回滚能力更不是忽略质量的理由。

1.2 为什么短代码不自动等于美

下面两段表达的业务意图近似，但认知成本不同：

CSHARP

```
var p = x.Where(i => i.S == 1 && !i.D)
    .Sum(i => i.Q * i.P * (1 - i.R));
```

CSHARP

```
var billableLines = lines.Where(line =>
    line.Status == LineStatus.Approved && !line.IsVoided);

var netAmount = billableLines.Sum(line =>
    line.Quantity * line.UnitPrice * (1m - line.DiscountRate));
```

本文综合建议：第二段更长，却让过滤条件、金额构成和 **decimal** 运算的意图更容易复核。第一段的短来自信息压缩，而不是复杂性消失。压缩格式、删测试、删除有价值注释、把类型变成字符串、把分支塞进三元运算，都可能降低 LOC，却把理解成本和风险转移给未来修改者。

.NET 的 CA1505 把 LOC 与程序体积、圈复杂度共同用于可维护性指数，并允许配置阈值或在有理由时抑制规则。[S5] Google 的工程实践也把“小变更”理解为一个自包含、聚焦的概念变更，而不是单纯按行数判断。[S13] 两者都不能支持“越短越好”或固定 LOC 目标。

1.3 审美判断的最小单位

本文综合建议：不要孤立评价一个方法。至少同时观察四个同心圆：

1. 表达式与方法：名称、控制流、失败语义是否清楚。
2. 类与模块：状态和变化是否有单一、可解释的拥有者。
3. 调用与数据边界：事务、网络、序列化、并发是否被保真。
4. 交付批次：变更能否独立验证、审查、回退和观察。

一个局部漂亮的方法若破坏数据库原子性，整体仍不美；一个模块边界若只能通过一次不可回退的全库重写得到，交付方式也损害了设计质量。这是本文综合建议，意在把代码审美扩展到代码在现实系统中的生命周期。

2. 信息隐藏：模块不是文件夹，而是变化的防火门

2.1 Parnas 的问题意识

Parnas 在 1972 年的原始论文中比较了不同的系统分解标准，并讨论模块化对灵活性、可理解性和开发时间的影响；其核心贡献是把模块边界与应被隐藏的设计决策联系起来。[S2] 这项研究提供的是分解原则与示例，不是现代 .NET 项目的固定目录模板，也不能推出“必须使用某种分层架构”。[S2]

把信息隐藏转成今天的工程问题，可以问：

- 哪个数据库表结构可能变化？谁应该知道它？
- 哪个外部供应商协议可能变化？谁不应知道它？
- 哪条金额、状态或权限规则由业务负责？谁能修改它？
- 哪种变化应当只修改一个模块，而不在控制器、服务和脚本中同步传播？

以上问题是本文综合建议，是对 Parnas 思路的教学性应用，而非论文原文。

2.2 按步骤切分不等于模块化

考虑以下目录：

TEXT

```
Invoices/
  Controllers/InvoiceController.cs
  Services/InvoiceService.cs
  Repositories/InvoiceRepository.cs
  Helpers/InvoiceHelper.cs
```

它有四层、四个文件，但若税率判断同时出现在 Controller、Service 和 SQL，数据库列名又穿透到 API DTO，这种分解只是把同一决策摊在更多位置。未来 修改税率适用规则仍需横跨所有层。

本文综合建议：真正的边界不是“有几个项目”，而是三件事：

1. 一个变化是否有明确拥有者；
2. 其他模块是否只通过稳定语义与它交互；
3. 边界是否阻止内部表示泄漏。

例如，`IInvoicePolicy.Calculate` 暴露“计算结果”而不暴露税表存储方式；

`IInventoryReservation.TryReserveAsync` 暴露“保留是否成功”而不暴露所用 SQL。接口名称并不自动创造边界；只有信息真的被隐藏、依赖方向真的被约束，边界才成立。这是本文综合建议。

2.3 从变化轴反推边界

本文综合建议：可用“独立变化轴”做一次简化建模。

变化轴	稳定语义	适合隐藏的细节
税费政策	计算应付金额	税率表、舍入顺序
库存保留	成功或不足	锁、SQL、存储引擎
通知投递	接受投递请求	HTTP、重试、供应商字段
报表取数	返回定义好的行模型	SQL、视图、索引提示

这张表不是要求“每行都建接口”。若一个细节稳定、纯本地、没有副作用，且不会独立变化，直接代码可能比抽象层更诚实。边界的成本包括命名、接线、导航、测试替身和兼容承诺；收益必须来自真实变化和风险，而非图形对称。

2.4 判断是否是真模块的四个实验

本文综合建议：对一个候选模块做四个思想实验。

1. 替换实验：存储或供应商改变，调用者是否基本不变？
2. 解释实验：新成员能否用一句业务语言说出模块责任？
3. 泄漏实验：调用者是否依赖内部表名、SDK 类型或配置键？
4. 故障实验：一个适配器失败时，故障面是否被边界限制并可观察？

若四个实验大多失败，先修正拥有关系和数据契约，再增加文件或项目。仅移动代码而不改变依赖，通常只是视觉整理。这一判断是本文综合建议。

3. 耦合、内聚与依赖方向

3.1 不要把耦合只理解成“引用数量”

.NET 的 CA1506 以一个类型引用的唯一类型数量来度量类耦合，并指出高耦合实体可能难以维护；该规则的阈值可配置，适用对象是 C# 与 Visual Basic 符号。[S4] 因而它适合发现热点，却不能单独判定某个类必须拆分。[S4]

本文综合建议：工程上至少区分四类耦合。

- 结构耦合：项目、包、命名空间和类型引用。
- 数据耦合：多个模块共同依赖一张宽表或一个“万能 DTO”。

- 时间耦合：调用必须按隐含顺序发生，例如先 `Init` 再 `Save`。
- 运行耦合：共享事务、网络可用性、全局缓存或同一消息语义。

一个类引用类型很少，也可能因共享数据库状态而高度耦合；另一个类引用很多 稳定值对象，却仍然职责内聚。因此，数字用于定位，调用与变化语义用于判断。这一结论是本文综合建议。

3.2 内聚不是“东西放在一起”

本文综合建议：高内聚意味着成员围绕同一业务责任共同变化，而不是因为“都和订单有关”就进入 `OrderHelper`。可用三个问题判断：

1. 这些方法是否维护同一组不变项？
2. 它们是否因同一种业务变化而一起修改？
3. 它们使用的数据是否属于同一生命周期？

例如，“计算订单可开票额”与“把发票 PDF 上传对象存储”都涉及发票，但前者是业务政策，后者是基础设施副作用。把它们放进同一个 `InvoiceService` 只是主题相似，不是内聚。

3.3 依赖反转、端口与适配器

Microsoft 的 .NET 架构指导用依赖反转解耦基础设施与其他层；其 Clean Architecture 说明让业务逻辑和应用模型位于核心，基础设施实现依赖应用核心，而不是核心反向依赖基础设施。[S3] 这是一种平台工程指导，不要求所有应用都复制同样的项目布局。[S3]

本文综合建议：“端口”是稳定策略需要的能力，“适配器”把具体技术翻译成该能力。方向可以表示为：

TEXT		
Application policy	--> INotificationPort	<-- HttpNotificationAdapter
stable need	port	volatile detail

箭头表示源码依赖：应用策略和 HTTP 适配器都依赖核心定义的端口。运行时由组合根把实现接入。端口应使用业务语言，例如 `SendShipmentNoticeAsync`，而不是把供应商 SDK 的 `PostVendorPayloadAsync` 暴露给核心。

3.4 稳定依赖方向不是“接口越多越好”

本文综合建议：下列情况更值得引入端口或抽象：

- 依赖产生网络、文件、数据库、时钟或随机数等副作用；
- 技术实现可能替换，而业务调用语义相对稳定；
- 测试需要稳定地制造成功、超时、拒绝或重复响应；
- 当前引用方向迫使稳定策略依赖易变细节；
- 边界承载安全、幂等、事务或可观察性约束。

本文综合建议：下列情况中，抽象可能只是仪式：

- 接口只有一个实现，且只是逐方法转发，没有隔离任何变化或副作用；
- 为一个纯函数建立接口、实现、工厂和 DI 注册，却没有替换需求；
- `IGenericService<T>` 用反射和字符串掩盖具体业务语义；
- 每个类都包装一次日志器或 ORM，调用者仍知道其全部细节；

- 接口随实现同步变化，真正稳定的契约并不存在。

判断标准不是“有几个实现”，而是抽象是否让依赖方向更诚实、局部推理更容易、风险更可测试。即使当前只有一个外部供应商，实现也可能值得有端口；即使有两个本地算法，直接策略函数也可能比 DI 容器更清楚。这是本文综合建议。

3.5 让边界可执行

ArchUnit 的维护者说明，该 Java 工具可分析字节码，并通过普通单元测试检查包、类、层、切片和循环依赖。[S10] 这只证明“架构边界可以写成可执行检查”这一工具能力；ArchUnit 不直接适用于 .NET，也不能替团队设计正确边界。[S10]

本文综合建议：.NET 项目可以使用适合自身技术栈的分析器、依赖图或架构测试库，从少数高价值规则开始，例如：

TEXT

Domain 不得引用 Infrastructure 命名空间。
Application 不得引用具体 HTTP SDK 类型。
模块外部不得引用 OtherModule.Internal 命名空间。

规则应有理由、负责人和例外审查机制。一次加入几十条噪声规则会诱发批量抑制，最终使护栏失去可信度。这是本文综合建议。

4. 重复的判断：相同文字不一定是相同行为

4.1 四种“相同”

PMD CPD 是复制粘贴检测器，能够定位重复代码块；其结果不能证明两个块在业务、事务、性能或异常语义上等价。[S11]

本文综合建议：看到重复时，依次区分：

1. 文字相同：令牌或语句长得一样。
2. 计算相同：相同输入在当前版本得到相同输出。
3. 业务知识相同：两处表达的是同一条规则，修订时应同步改变。
4. 变化方向相同：它们不仅今天相同，而且由同一责任人、同一原因驱动。

只有第三、第四层同时成立时，合并的收益通常最可靠。若两处折扣计算分别属于零售促销和长期合同，即使今天公式都是 $\text{amount} * 0.9m$ ，未来也很可能由不同政策变化。保留两处清晰的小函数，可能比制造 DiscountEngine 更美。

4.2 有意重复是债务，还是边界成本

本文综合建议：可以保留有意重复，但应能回答：

- 两处的业务拥有者或变化原因是否不同？
- 抽象后是否会产生跨模块依赖？
- 新名字能否准确表达共同概念？
- 合并是否迫使调用者传入开关、委托或配置来恢复差异？
- 故障是否会从一个模块扩散到多个模块？
- 重复是否小、稳定、易读且容易各自测试？

保留时可写一条短注释说明“相似但独立变化”，并在评审记录中保留决策依据。注释不应为糟糕实现辩护，而应防止未来开发者误把边界当重复。这是本文综合建议。

4.3 为什么布尔开关万能函数常常更差

CSHARP

```
decimal Calculate(  
    Invoice invoice,  
    bool includeTax,  
    bool useLegacyRounding,  
    bool isExport,  
    bool ignoreMinimum)
```

本文综合建议：四个布尔参数理论上产生 16 种组合，其中很多组合可能无意义或非法。调用 `Calculate(x, true, false, true, false)` 时，意图不可见；每增加一个差异，分支、测试组合和错误空间都会扩大。更好的方向可能是明确命名的策略或用例：

CSHARP

```
domesticPolicy.Calculate(invoice);  
exportPolicy.Calculate(invoice);  
legacyPreview.Calculate(invoice);
```

这不表示“禁止所有 bool”。`TryParse(..., ignoreCase: true)` 之类局部、单一、命名参数清楚的选择很普通。危险信号是用多个开关把本应独立变化的业务策略塞回一个函数。这个判断是本文综合建议。

4.4 去重决策记录

本文综合建议：对非显然的去重，用六行记录即可：

TEXT

```
候选：零售导入校验 / 合同导入校验  
相同：必填、日期格式的局部语法规则  
不同：客户匹配、税务状态、错误聚合方式  
变化轴：分别由零售运营与合同管理负责  
决定：只复用无业务含义的日期解析；保留编排重复  
证据：两套特征测试、历史变更记录、模块负责人确认
```

记录的目的是不是增加流程，而是让下一位维护者知道“这是经过判断的重复”，避免反复合并再拆分。这是本文综合建议。

5. 删除死代码与兼容层：静态沉默不等于运行时不存在

5.1 静态候选为什么不够

.NET 的裁剪分析从已知入口沿编译期可见路径追踪；反射和运行时装载等动态模式会让静态分析无法确定运行时访问目标。[S7] Microsoft 对裁剪警告的指导指出，警告可能意味着行为改变或崩溃，存在警告时应充分测试裁剪后的应用；动态模式与抑制需要谨慎处理。[S8]

这些文档讨论的是 .NET 发布裁剪，不是任意项目死代码分析的完备理论。[S7] [S8] 但它们足以提醒我们：
`Find All References = 0`、分析器“未使用”、IDE 变灰，都只能产生候选，不能单独授权删除。

5.2 九类隐形入口

本文综合建议：删除候选前，至少核查以下入口。

1. 反射：`Type.GetType`、`GetMethod`、特性发现、表达式编译。
2. DI 扫描：按接口、特性、命名约定或程序集扫描自动注册。

3. 配置：类型名、处理器名、路由名、功能开关或程序集路径。
4. 序列化：JSON/XML 字段、类型判别器、旧字段别名、模型绑定。
5. 插件：外部程序集、脚本宿主、运行时模块目录。
6. 调度：Quartz/后台服务/数据库任务表中的 Job 类型与任务键。
7. 脚本：PowerShell、Shell、SQL、部署流水线、运维手册里的入口。
8. 公开契约：NuGet 公共类型、HTTP 路由、消息主题、数据库对象。
9. 生成与原生互操作：Source Generator、P/Invoke、COM、AOT 保留标记。

每一类都可能把“源码没有直接引用”的类型变成真实入口。核查范围应按系统能力裁剪，但不能因为搜索麻烦就把未知当作不存在。这是本文综合建议。

5.3 删除证据阶梯

本文综合建议：可把删除授权分成六级，风险越高，所需级别越高。

级别	证据	能说明什么	仍不能说明什么
E1	文本与引用搜索	无显式源码引用	无动态入口
E2	构建与静态分析	编译路径不需要	运行配置不需要
E3	配置/契约清单	已核对已知入口	未观测环境无调用
E4	特征与集成测试	覆盖场景行为不变	生产全量都无调用
E5	运行遥测与审计	观测窗内无调用	低频年度场景不存在
E6	所有者确认与回退	业务接受且可恢复	永久零风险

例如，删除内部格式化帮助方法可能以 E1-E4 足够；删除旧财务消息字段可能需要 E1-E6，并覆盖月末、年末或外部合作方的低频场景。证据不是“越多越好”，而是与损失严重度、调用稀有度和恢复难度相称。这是本文综合建议。

5.4 兼容代码的四种状态

本文综合建议：不要把所有旧代码都叫“垃圾”。给候选标注状态：

- **ACTIVE_COMPATIBILITY**：仍有被支持的调用方。
- **DEPRECATION_WINDOW**：已公告替代方案，正在计时退出。
- **UNCONFIRMED**：证据不足，暂不删除并明确缺口。
- **REMOVABLE**：契约、运行和回退证据满足删除条件。

兼容层若保留，应写清调用方、截止条件、监测方式和责任人；若删除，应把旧契约验证改成“明确拒绝”或“迁移完成”的测试，而不是只删实现。这是本文综合建议。

5.5 低频并不等于无价值

本文综合建议：调度任务、年度结转、灾备脚本和失败补偿可能长期不运行，却在少数时刻承担高价值责任。只看 30 天日志会误删季末或年度入口。观测窗应覆盖业务周期；无法覆盖时，应保留未知状态并寻求所有者和部署证据。

6. 行为保持重构：把“没有改功能”写成可检验命题

6.1 什么叫行为

Fowler 将重构描述为一系列小的、保持行为的转换，并强调小步降低错误风险、使系统保持可工作。[S12] 但“行为”在业务系统中不能只理解为方法返回值。

本文综合建议：重构前至少枚举这些不变项：

- 接口：路由、状态码、字段名、空值、排序、错误码。
- 业务：金额、舍入、状态迁移、权限、校验优先级。
- 数据：写入行数、唯一性、默认值、审计字段、历史兼容。
- 事务：原子性、隔离要求、失败后的提交或回滚边界。
- 并发：竞争条件、重复请求、锁与幂等语义。
- 外部副作用：消息次数、顺序、重试、文件与网络调用。
- 时间与性能：时区、截止时刻、关键路径可接受区间。
- 可观察性：关联 ID、关键日志、指标和审计信息。

已知缺陷也属于基线：若本批次是纯重构，先标为“暂时保持”；若要修复，另建 功能/缺陷批次。Google 的工程实践建议让较大的重构与功能或缺陷修改分开，并让纯重构也受到相关测试覆盖。[S13]

6.2 基线不是“测试都绿”一句话

Microsoft 的 ASP.NET Core 测试指导区分单元、集成和功能测试，并强调可失败 条件与关键业务逻辑，而不是单纯追逐覆盖率数字。[S6] Microsoft 的 Well-Architected 测试指导还强调验证测试自身，并让回归集优先包含高价值且 稳定的测试。[S9]

本文综合建议：一份可用基线应记录命令、环境、输入、预期、实际和已知 缺口。例如：

TEXT

```
Build: dotnet build Billing.sln -c Release
Unit: dotnet test tests/Billing.UnitTests/Billing.UnitTests.csproj
Integration: dotnet test tests/Billing.IntegrationTests/...
Scenario: POST /invoices/preview, fixture INV-ROUND-005
Data: InvoiceLine count and NetAmount before/after equal
External: notification fake received exactly one idempotency key
Known gap: real provider timeout recovery not exercised locally
```

命令只是证据入口，输出摘要或制品位置也应保存。一次通过的测试不等于生产 证明；它说明的是特定版本、环境和输入下，已声明断言成立。这是本文综合建议。

6.3 小而可逆的批次

Google 的工程实践把小变更定义为聚焦于一个自包含概念，而非固定行数；它还建议重构与功能/缺陷工作通常分开。[S13]

本文综合建议：一个理想重构批次可以用一句动词短语描述，例如：

- “提取金额舍入政策，不改变调用或持久化”；
- “用通知端口包围现有 HTTP 客户端，暂不切换供应商”；
- “迁移唯一一个动态 Job 配置后删除旧 Job 类型”。

每批都遵循：

TEXT

识别 -> 理解 -> 包围 -> 替换 -> 对比 -> 切换 -> 验证 -> 删除 -> 记录

这里的“包围”指先用特征测试、适配器或观测点把旧行为固定；“对比”可让新旧 实现对同一输入产生可比较结果；“删除”发生在切换和验证之后。该流程是本文 综合建议，不是任何来源逐字规定。

6.4 测试层与风险相配

本文综合建议：测试层次应对应失败机制。

风险	主要测试	必要补充
纯金额规则	单元/性质测试	边界样例、舍入表
ORM 映射/SQL	集成测试	实际数据库或兼容环境
HTTP 契约	功能/契约测试	超时、重试、幂等替身
库存并发	并发集成测试	数据约束、事务观测
序列化兼容	快照/契约测试	旧消费者样例
调度入口	配置启动测试	时间窗、部署清单

覆盖率能提示“哪些代码从未被测试触达”，却不能证明断言正确、事务语义正确 或关键行为被覆盖。Microsoft 的测试指导明确把关键业务路径和可能失败条件置于 覆盖率数字之前。[S6]

6.5 回滚是设计输入

本文综合建议：在修改前回答：

1. 回滚单位是一个提交、一个部署包、一个配置开关，还是数据迁移？
2. 新版本写出的数据，旧版本是否还能读？
3. 外部副作用发生后，代码回滚能否撤销它？
4. 需要前向修复而不是回滚的场景是什么？
5. 由谁判断暂停、回滚或继续？依据哪条指标？

数据库不可逆迁移、已发出的消息和外部扣款不能靠 `git revert` 撤回。高风险 批次应先设计兼容读写、幂等补偿或分阶段切换。这里是本文综合建议，具体策略 必须接受项目和业务所有者审查。

6.6 观察行为差异

本文综合建议：重构不要移除诊断所需的可观察性。至少保留或改善：

- 一次业务请求贯穿日志、数据库和外部调用的关联标识；
- 关键结果数量与失败分类，而不是记录敏感负载；
- 新旧路径切换比例与差异计数；
- 重试、超时、幂等冲突和回滚次数；
- 能指向版本与配置的部署标识。

可观察性也要有边界：不得为了调试泄露凭据、个人信息或完整业务报文。何种 字段可记录应遵循系统自身安全与合规政策。这是本文综合建议。

7. 指标：测量地图，不要奖励绕路

7.1 正确理解五类指标

ISO/IEC 25010:2023 的公开页面说明产品质量模型可用于指定、度量与评价等活动，但公开元数据不提供 LOC 目标，也不能据此推出具体阈值。[S1] ISO/IEC 5055:2021 的公开摘要把自动化源代码质量度量与检测、计数架构和编码实践违规及其潜在运行风险或成本联系起来；公开页不展示标准正文，本文不主张其具体算法或阈值。[S14]

本文综合建议：五类常见指标各回答不同问题。

- LOC：代码体量趋势如何？不回答代码是否正确。
- 复杂度：哪些控制流更难穷举？不回答业务是否应该复杂。
- 耦合：哪些实体依赖广？不回答依赖是否合理或运行耦合多强。
- 重复：哪些文本可能同步维护？不回答业务知识是否相同。
- 覆盖率：哪些代码被测试执行过？不回答断言质量和风险覆盖。

CA1505 把 LOC 与程序体积、圈复杂度组合，并允许配置阈值。[S5] CA1506 统计唯一类型引用来提示类耦合，阈值同样可配置。[S4] CPD 可以定位复制粘贴块，却不证明语义等价。[S11] 因而单个数字最多是调查入口。

7.2 一个紧凑的瘦身记分卡

下表是本文综合建议，权重体现“行为与风险优先于规模”。每项按 0-4 分评分，分数越高越好；权重只用于一个项目内前后对比，不用于跨团队排名。

维度	权重	0 分与 4 分锚点	证据
行为保持	30	未知 / 关键不变项全通过	分层测试、数据对比
风险降低	25	风险扩大 / 故障面受控	依赖、事务、回滚
边界质量	15	反向依赖 / 变化被隐藏	架构检查、评审
可验证性	15	不能重现 / 稳定可重现	测试与验收清单
可观察与可逆	10	盲区 / 可定位可恢复	日志、指标、演练
规模与热点	5	恶化 / 合理改善	LOC、复杂度、重复

总分计算示例：

TEXT
Score = sum(item_score / 4 * weight) Gate = 行为保持 >= 3 且无未接受的高严重程度回归

即使总分提高，只要行为保持低于门槛，批次也不能判为成功。相反，为了增加特征测试、明确类型或适配器而多出少量代码，若显著改善风险与验证能力，可能是更美的结果。这是本文综合建议。

7.3 防止指标游戏

本文综合建议：记录指标时遵循五条纪律。

1. 生产、测试、生成物、第三方代码分别统计。
2. 报告绝对值、变化量、排除范围和工具版本。
3. 指标下降要能关联到具体风险或认知负担的下降。

4. 不为降低 LOC 删除测试、注释、类型或错误处理。
5. 先固定行为门槛，再看趋势；不得用综合分掩盖关键回归。

若一个团队被奖励“重复率下降 50%”，最容易的做法可能是制造一个巨型万能函数；若被奖励“覆盖率 90%”，最容易的做法可能是执行大量代码却没有关键断言。指标设计必须让正确行为比绕过数字更容易。这是本文综合建议。

第二部分 八个 C# 判断案例

以下代码均为原创、通用、脱敏的教学示例。它们聚焦设计形状，省略项目专有 认证、日志和 ORM 细节；“概念上可编译”不等于复制后可直接上线。每个案例 都明确列出气味、推理、改进或不改的理由，以及安全证据。

8. 案例一：发票金额 - 把政策从流程噪声中显露出来

8.1 改前

CSHARP

```
public decimal GetAmount(Invoice invoice)
{
    var total = 0m;

    foreach (var line in invoice.Lines)
    {
        if (!line.Deleted && line.Status == 2)
        {
            var value = line.Quantity * line.Price;
            value -= value * line.Discount;
            total += Math.Round(value * 1.13m, 2);
        }
    }

    return total < 0m ? 0m : total;
}
```

本文综合建议 - 气味：2 和 1.13m 的含义隐藏；折扣率边界、含税 规则、逐行舍入还是汇总舍入、负数归零是否真实业务规则都无法从签名判断。方法把筛选、金额政策和容错混在一起，因此一个“简化循环”的重构很可能 意外改变舍入顺序。

8.2 推理

本文综合建议：先写不变项，而不是先提取方法。

TEXT

只计算 Approved 且未作废的行。
每行先计算折后净额，再计税并以 ToEven 舍入到两位，最后求和。
货币使用 decimal；税率、舍入位置和舍入模式不得隐式变化。
本结构批次保留旧逻辑对越界折扣的处理和总额负数归零。

这里最重要的设计决定不是类的数量，而是把“舍入顺序”变成可见政策。若现有 系统实际上逐行舍入，纯重构就必须先保持逐行舍入；改变到汇总舍入应另建 需求批次。同样，拒绝越界折扣和取消负数归零都属于行为变化，不能伪装成 命名或提取方法。这是本文综合建议。

8.3 改后

CSHARP

```
public enum InvoiceLineStatus
{
    Unknown = 0,
    Approved = 2
}

public sealed record InvoiceLine(
    decimal Quantity,
    decimal UnitPrice,
    decimal DiscountRate,
    InvoiceLineStatus Status,
    bool IsVoided);

public sealed class InvoiceAmountPolicy
{
    private const decimal TaxRate = 0.13m;

    public decimal Calculate(ICollection<InvoiceLine> lines)
    {
        var total = lines
            .Where(IsBillable)
            .Sum(CalculateRoundedLineAmount);

        return total < 0m ? 0m : total;
    }

    private static bool IsBillable(InvoiceLine line) =>
        line.Status == InvoiceLineStatus.Approved && !line.IsVoided;

    private static decimal CalculateRoundedLineAmount(InvoiceLine line)
    {
        var gross = line.Quantity * line.UnitPrice;
        var discounted = gross * (1m - line.DiscountRate);
        var amountWithTax = discounted * (1m + TaxRate);

        return decimal.Round(
            amountWithTax,
            decimals: 2,
            mode: MidpointRounding.ToEven);
    }
}
```

本文综合建议 - 改进理由：领域枚举把已知状态码 2 显式固定为 Approved，没有为其他未知码虚构名称；政策只依赖行值，不依赖 数据库或 HTTP；逐行舍入的位置和模式显式。越界折扣与负数归零仍是可疑的 遗留行为，但本批次有意保留并明确标注，避免把功能修正混入结构重构。代码 行数增加了，但意图、不变项、局部推理和可测试性同时改善。

8.4 安全证据

本文综合建议 - 证明包：

1. 用生产脱敏样例建立改前“黄金结果”，覆盖半分舍入、空集合和作废行。
2. 加入两行 **Quantity = 1m**、**UnitPrice = 0.005m**、零折扣的回归样例；旧、新 实现都应返回 **0.02m**，从而捕获误改为汇总舍入的实现。
3. 用表格测试覆盖折扣 **0**、**1**、边界外值、负数总额、状态码 **2** 与未知状态，确认遗留行为仍一致；不要在测试中悄悄把输入域缩成“只有合法折扣”。
4. 运行新旧实现对比，确认每张发票及汇总结果一致。
5. 若持久化金额，集成测试应验证数据库精度、比例和 **NULL** 处理。
6. 只有在产品确认舍入、折扣与负数规则后，才把潜在旧缺陷转为单独的行为 变更。

Microsoft 的测试指导支持优先测试关键业务逻辑与可失败条件，而不是只看覆盖率。[S6] 它不能替本项目确认具体税率或舍入政策。

9. 案例二：库存保留 - 美感不能以并发正确性为代价

9.1 改前

CSHARP

```
public async Task<bool> ReserveAsync(
    Guid itemId,
    int quantity,
    CancellationToken cancellationToken)
{
    var item = await _db.Inventory
        .SingleAsync(x => x.Id == itemId, cancellationToken);

    if (item.AvailableQuantity < quantity)
    {
        return false;
    }

    item.AvailableQuantity -= quantity;
    await _db.SaveChangesAsync(cancellationToken);
    return true;
}
```

本文综合建议 - 气味：代码看起来线性、短、易读，但“读取数量”和“写回 数量”是两个可竞争步骤。两个请求可能同时读到足够库存并都成功扣减。局部 美感掩盖了运行耦合与并发不变项。

9.2 推理

本文综合建议：库存核心不变项是“可用量不得因竞争而小于零，且成功保留 必须与保留记录处于同一业务事务”。这个约束必须由能序列化竞争的数据库操作 保证，而不是依赖进程内锁或“先读再检查”。多实例部署、重启和旁路写入都会 使进程内保护失去完整性。

9.3 改后

应用层端口只表达业务能力：

CSHARP

```
public interface IInventoryReservationStore
{
    Task<bool> TryReserveAsync(
        Guid itemId,
        int quantity,
        Guid reservationId,
        CancellationToken cancellationToken);
}

public sealed class ReserveInventory
{
    private readonly IInventoryReservationStore _store;

    public ReserveInventory(IInventoryReservationStore store)
    {
        _store = store;
    }

    public Task<bool> ExecuteAsync(
        Guid itemId,
        int quantity,
        Guid reservationId,
        CancellationToken cancellationToken)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(quantity);

        return _store.TryReserveAsync(
            itemId,
            quantity,
            reservationId,
            cancellationToken);
    }
}
```

适配器在同一个数据库事务内使用条件原子更新，并写入具有唯一索引键的 保留记录。下面是概念性 SQL，参数必须由数据库驱动绑定：

SQL

```
UPDATE Inventory
SET AvailableQuantity = AvailableQuantity - @quantity
WHERE ItemId = @itemId
AND AvailableQuantity >= @quantity;

-- 仅当上一步影响 1 行时，在同一事务插入 Reservation。
-- ReservationId 应有唯一约束，以约束重复请求。
```

9.3.1 适配器关键形状

CSHARP

```
public async Task<bool> TryReserveAsync(
    Guid itemId,
    int quantity,
    Guid reservationId,
    CancellationToken cancellationToken)
{
    await using var transaction =
        await _db.Database.BeginTransactionAsync(cancellationToken);

    var changed = await ExecuteConditionalDecrementAsync(
        itemId,
        quantity,
        cancellationToken);

    if (changed != 1)
    {
        await transaction.RollbackAsync(cancellationToken);
        return false;
    }

    await InsertReservationAsync(
        reservationId,
        itemId,
        quantity,
        cancellationToken);

    await transaction.CommitAsync(cancellationToken);
    return true;
}
```

本文综合建议 - 改进理由：端口隐藏存储策略，但不能把事务事实隐藏到没人验证的黑盒里。实现必须以单条条件更新处理竞争，以同一事务绑定扣减与记录，以唯一约束处理幂等。若具体 ORM 的执行与事务 API 不同，应按实际驱动验证，不得照抄示例签名。

9.4 安全证据

本文综合建议 - 证明包：

1. 在与生产兼容的数据库上并发发起超过库存的保留请求。
2. 断言成功数量之和不超过初始库存，最终库存不为负。
3. 断言每个成功响应都有一条保留记录，失败响应没有孤儿记录。
4. 用同一 `reservationId` 重试，断言不发生重复扣减。
5. 在插入保留记录时注入失败，断言扣减回滚。
6. 记录数据库隔离级别、约束和执行计划；本地纯单元测试不足以证明原子性。

这里的验证策略是本文综合建议。静态代码整洁不能替代真实数据库并发证据。

10. 案例三：通知客户端 - 用端口隔离副作用，不泄漏供应商语言

10.1 改前

CSHARP

```
public sealed class ShipmentService
{
    private readonly HttpClient _client;

    public ShipmentService(HttpClient client)
    {
        _client = client;
    }

    public async Task ShipAsync(
        Shipment shipment,
        CancellationToken cancellationToken)
    {
        shipment.MarkShipped();
        await _repository.SaveAsync(shipment, cancellationToken);

        var body = new
        {
            template_code = "TPL_908",
            mobile = shipment.Phone,
            args = new[] { shipment.Number }
        };

        await _client.PostAsJsonAsync(
            "/vendor/send",
            body,
            cancellationToken);
    }
}
```

本文综合建议 - 气味：应用服务知道供应商字段、模板编码和 URL；状态写入与网络失败的关系不清；HTTP 非成功状态若未检查，可能被当成通知成功。测试 发货规则必须同时处理网络。

10.2 推理

Microsoft 的 .NET 架构指导支持用依赖反转让基础设施依赖应用核心定义的 抽象。[S3] 它并不要求“每个类一个接口”。这里值得引入端口，是因为通知是 外部副作用、失败模式可独立变化、测试需要制造拒绝与超时，而业务只关心“发货通知是否被接受”。这段取舍是本文综合建议。

10.3 改后

CSHARP

```
public sealed record ShipmentNotice(
    string ShipmentNumber,
    string Recipient,
    string IdempotencyKey);

public enum NoticeOutcome
{
    Accepted,
    Rejected,
    RetryableFailure
}

public interface IShipmentNoticePort
{
    Task<NoticeOutcome> SendAsync(
        ShipmentNotice notice,
        CancellationToken cancellationToken);
}
```

CSHARP

```
public sealed class VendorShipmentNoticeAdapter
    : IShipmentNoticePort
{
    private readonly HttpClient _client;

    public VendorShipmentNoticeAdapter(HttpClient client)
    {
        _client = client;
    }

    public async Task<NoticeOutcome> SendAsync(
        ShipmentNotice notice,
        CancellationToken cancellationToken)
    {
        var payload = new
        {
            template_code = "TPL_908",
            mobile = notice.Recipient,
            args = new[] { notice.ShipmentNumber },
            request_id = notice.IdempotencyKey
        };

        using var response = await _client.PostAsJsonAsync(
            "/vendor/send",
            payload,
            cancellationToken);

        if (response.IsSuccessStatusCode)
        {
            return NoticeOutcome.Accepted;
        }

        return IsRetryable(response.StatusCode)
            ? NoticeOutcome.RetryableFailure
            : NoticeOutcome.Rejected;
    }

    private static bool IsRetryable(HttpStatusCode statusCode) =>
        statusCode == HttpStatusCode.TooManyRequests ||
        (int)statusCode >= 500;
}
```

应用策略使用领域语言：

CSHARP

```
var notice = new ShipmentNotice(
    shipment.Number,
    shipment.Phone,
    shipment.Id.ToString("N"));

var outcome = await _noticePort.SendAsync(
    notice,
    cancellationToken);

return outcome switch
{
    NoticeOutcome.Accepted => ShipmentResult.Notified,
    NoticeOutcome.RetryableFailure => ShipmentResult.PendingRetry,
    _ => ShipmentResult.NotificationRejected
};
```

本文综合建议 - 改进理由：核心不再依赖 HTTP 或供应商 DTO；适配器负责 翻译协议；失败分类成为显式契约；幂等键可被测试和观察。至于状态保存、消息 发送采用 Outbox、同步补偿还是允许“已发货待通知”，必须由真实业务一致性 要求决定，不能由这个示例替项目决定。

10.4 安全证据

本文综合建议 - 证明包：

1. 端口单元测试覆盖 Accepted、Rejected、RetryableFailure 三条策略路径。
2. 适配器契约测试检查方法、URL、字段名、鉴权和非成功响应映射。
3. 重试测试断言幂等键不变、成功后不再重复发送。
4. 集成场景断言发货状态与通知失败状态符合已确认的一致性政策。
5. 发布后观察供应商状态分布、超时、重试和重复拒绝，不记录完整手机号。

11. 案例四：导入校验 - 保留有意重复，拒绝布尔万能入口

11.1 改前的“去重”提案

CSHARP

```
public ImportResult Validate(
    ImportRow row,
    bool isContract,
    bool allowMissingCustomer,
    bool stopOnFirstError,
    bool useLegacyDate)
{
    // 多种导入规则和错误策略交错。
    throw new NotImplementedException();
}
```

本文综合建议 - 气味：这不是原始重复，而是一次准备合并的设计。四个布尔值把合同导入与零售导入的不同业务拥有者、错误聚合方式和兼容周期压进一个入口。调用方难以表达意图，非法组合也无法被类型系统排除。

11.2 推理

CPD 发现的是复制粘贴块，不证明两套导入在业务上应共同变化。[S11]

本文综合建议：调查后假设得到如下事实：两者都要解析 ISO 日期、检查空单元格；合同导入需要累计所有错误供批量修复，零售导入遇到客户不存在时立即停止；两套规则由不同团队维护。共同知识只有“无业务含义的日期语法”，编排与错误政策不是同一知识。

11.3 改后：只复用稳定的小核

CSHARP

```
public static class ImportDateParser
{
    public static bool TryParse(
        string? value,
        out DateOnly date)
    {
        return DateOnly.TryParseExact(
            value,
            "yyyy-MM-dd",
            CultureInfo.InvariantCulture,
            DateTimeStyles.None,
            out date);
    }
}
```

CSHARP

```
public sealed class ContractImportValidator
{
    public IReadOnlyList<ImportError> Validate(ContractRow row)
    {
        var errors = new List<ImportError>();

        if (string.IsNullOrEmpty(row.ContractNumber))
        {
            errors.Add(new("ContractNumber", "Required"));
        }

        if (!ImportDateParser.TryParse(row.StartDate, out _))
        {
            errors.Add(new("StartDate", "InvalidDate"));
        }

        return errors;
    }
}
```

CSHARP

```
public sealed class RetailImportValidator
{
    public ImportDecision Validate(
        RetailRow row,
        IReadOnlySet<string> knownCustomers)
    {
        if (!knownCustomers.Contains(row.CustomerCode))
        {
            return ImportDecision.Stop("UnknownCustomer");
        }

        if (!ImportDateParser.TryParse(row.SaleDate, out _))
        {
            return ImportDecision.Reject("InvalidDate");
        }

        return ImportDecision.Accept();
    }
}
```

本文综合建议 - 改进理由：两个校验器仍有表面相似的 `if`，这是有意重复。它们的输出类型和控制流忠实表达不同错误政策；只有无业务含义且稳定的日期解析被共享。这里“没有完全去重”正是更好的设计决定。

11.4 安全证据

本文综合建议 - 证明包：

1. 分别用真实脱敏导入样例建立特征测试，保留错误顺序和聚合语义。
2. 测试日期解析边界：闰日、空值、区域性格式和非法尾随字符。
3. 检查调用点，确认没有布尔组合被遗漏。
4. 由两类导入的业务所有者分别确认错误政策。
5. 在决策记录中写明“只共享语法，不共享业务编排”。

12. 案例五：报表查询 - 让数据粒度、空值和排序进入契约

12.1 改前

CSHARP

```
public async Task<List<dynamic>> GetReportAsync(
    string customerCode,
    DateTime from,
    DateTime to)
{
    var sql = $$$$
    SELECT CustomerCode, SUM(Amount) Amount
    FROM Invoice
    WHERE CustomerCode = '{customerCode}'
    AND InvoiceDate >= '{from:yyyy-MM-dd}'
    AND InvoiceDate < '{to:yyyy-MM-dd}'
    GROUP BY CustomerCode
    $$$;

    return await _db.QueryAsync<dynamic>(sql);
}
```

本文综合建议 - 气味：字符串插值引入注入与区域格式风险；**dynamic** 隐藏 结果契约；**to** 是包含还是排除不清；无发票客户是“不返回行”“金额 0”还是 **NULL** 未定义；没有排序保证。把 SQL 提取到文件不会自动解决这些问题。

12.2 推理

本文综合建议：报表重构前先固定四件事：

1. 粒度：一行是客户、客户加币种，还是客户加月份？
2. 集合：只含有发票客户，还是所有客户即使金额为零？
3. 值语义：缺失是 **NULL**、0、空字符串，还是不返回行？
4. 顺序：消费者是否依赖稳定排序和合计行位置？

如果这些未定义，任何“更整洁”的 LINQ 或 SQL 都可能改变报表含义。

12.3 改后

CSHARP

```
public sealed record InvoiceSummaryQuery(
    string CustomerCode,
    DateOnly FromInclusive,
    DateOnly ToExclusive);

public sealed record InvoiceSummaryRow(
    string CustomerCode,
    decimal Amount);

public interface IInvoiceSummaryReader
{
    Task<IReadOnlyList<InvoiceSummaryRow>> QueryAsync(
        InvoiceSummaryQuery query,
        CancellationToken cancellationToken);
}
```

适配器使用参数化 SQL，并把顺序写进查询：

CSHARP

```
const string Sql = ""
SELECT
    c.CustomerCode,
    COALESCE(SUM(i.Amount), 0) AS Amount
FROM Customer c
LEFT JOIN Invoice i
    ON i.CustomerCode = c.CustomerCode
    AND i.InvoiceDate >= @fromInclusive
    AND i.InvoiceDate < @toExclusive
WHERE c.CustomerCode = @customerCode
GROUP BY c.CustomerCode
ORDER BY c.CustomerCode
"";

var parameters = new
{
    customerCode = query.CustomerCode,
    fromInclusive = query.FromInclusive,
    toExclusive = query.ToExclusive
};

return await connection.QueryAsync<InvoiceSummaryRow>(
    new CommandDefinition(
        Sql,
        parameters,
        cancellationToken: cancellationToken));
```

本文综合建议 - 改进理由：查询对象命名日期边界，结果类型固定粒度，SQL 参数化，**LEFT JOIN** 与 **COALESCE** 明示“所有目标客户且无发票为 0”，**ORDER BY** 明示稳定顺序。若真实需求是“不返回无发票客户”或保留 **NULL**，就不应照抄此查询；应让类型、SQL 和验收样例共同表达真实契约。

12.4 安全证据

本文综合建议 - 证明包：

1. 在实际数据库兼容环境执行改前与改后查询，参数集相同。
2. 对比行数、键集合、金额、**NULL**/0、排序与尾部合计行。
3. 覆盖无发票、跨日边界、多币种、冲销、重复联接和大金额精度。
4. 检查执行计划与代表性数据性能，不把“结果相同”误作“风险全同”。
5. API 或导出层测试应验证列名、格式与消费者所见顺序。

13. 案例六：调度任务 - 零引用类型可能是生产入口

13.1 删除候选

CSHARP

```
public sealed class RebuildMonthlySnapshotJob : IJob
{
    public Task Execute(IJobExecutionContext context)
    {
        return RebuildAsync(context.CancellationToken);
    }
}
```

IDE 显示该类零引用，开发者准备删除。

本文综合建议 - 气味：这里真正的气味不是类本身，而是调度入口依赖类型名 字符串且没有受管清单。Quartz、DI 扫描或任务数据库可能通过反射创建该类型。.NET 裁剪文档说明反射与运行时装载会超出静态可达性分析能可靠确定的范围。[S7][S8]

13.2 推理与安全迁移

本文综合建议：先搜索源码、配置仓库、部署变量、任务表、运维脚本与运行日志。假设发现任务表仍有：

TEXT

```
JobKey: monthly-snapshot
HandlerType: Jobs.RebuildMonthlySnapshotJob, Billing.Jobs
Cron: 0 0 2 1 * ?
```

它每月只运行一次，最近 14 天无日志毫无证明力。不能删除。先把动态类型名 迁移为稳定业务键和显式注册：

CSHARP

```
public static class JobRegistry
{
    public static IReadOnlyDictionary<string, Type> Handlers { get; } =
        new Dictionary<string, Type>
        {
            ["monthly-snapshot"] =
                typeof(RebuildMonthlySnapshotJob)
        };
}
```

CSHARP

```
var handlerType = JobRegistry.Handlers.GetValueOrDefault(job.HandlerKey)
?? throw new InvalidOperationException(
    $"Unknown job handler: {job.HandlerKey}");
```

本文综合建议 - 改进理由：显式注册把隐形入口变成可搜索、可测试的清单，未知键会明确失败。这里暂时不删除 Job，因为迁移尚未完成。这是“有证据的 不改”，比零引用清理更符合审美。

只有当任务已迁移或业务决定下线，旧类型才进入 **DEPRECATION_WINDOW**，并在 覆盖完整业务周期、确认无旧配置后删除。具体观测窗取决于调度周期与损失风险，不是固定天数。这是本文综合建议。

13.3 安全证据

本文综合建议 - 证明包：

1. 导出所有环境的任务键、处理器与启停状态，不能只查开发环境。
2. 启动测试遍历任务表，断言每个键都能解析且 DI 能构造。
3. 在受控环境手动触发任务，验证数据、幂等和失败重试。
4. 观测至少覆盖一个真实调度周期，并核查低频月末/年末变体。
5. 删除前保留可恢复提交与旧配置备份；删除后未知旧键应清晰报警。

14. 案例七：序列化兼容 - 内部重命名不应偷改线缆协议

14.1 改前

CSHARP

```
public sealed class InvoiceRequest
{
    public string CustomerId { get; init; } = string.Empty;
    public decimal Amount { get; init; }
}
```

开发者把 **CustomerCode** 重命名为 **CustomerId**，所有 C# 引用均已编译通过。

本文综合建议 - 气味：若 API 默认以属性名生成 JSON，内部重命名可能把 线缆字段从 `customerCode` 改成 `customerId`。源码调用全通过也不能证明旧 客户端、脚本或消息消费者仍兼容。序列化本身就是动态/契约入口之一。

14.2 改后：把线缆契约与领域名称分开

CSHARP

```
public sealed class InvoiceRequestContract
{
    [JsonPropertyName("customerCode")]
    public string CustomerCode { get; init; } = string.Empty;

    [JsonPropertyName("amount")]
    public decimal Amount { get; init; }
}

public sealed record CreateInvoiceCommand(
    CustomerId CustomerId,
    decimal Amount);
```

CSHARP

```
public static class InvoiceRequestMapping
{
    public static CreateInvoiceCommand ToCommand(
        InvoiceRequestContract contract)
    {
        var customerId = CustomerId.Parse(contract.CustomerCode);
        return new CreateInvoiceCommand(customerId, contract.Amount);
    }
}
```

本文综合建议 - 改进理由：外部 DTO 明确固定线缆字段，领域命令可以采用 更准确的内部类型名，映射处承担翻译与校验。接口没有“消灭变化”，而是把 外部兼容变化与内部模型变化隔离。

如果系统同时接收历史字段 `custCode`，应采用经过评审的兼容解析方案，并对 冲突字段定义明确优先级和错误，而不是静默猜测。只有在调用方清单、遥测和 公告窗口满足退出条件后才能删除旧字段。这是本文综合建议。

14.3 安全证据

本文综合建议 - 证明包：

1. 契约测试序列化并断言精确 JSON 字段、大小写、空值与数值格式。
2. 反序列化旧客户端样例，确认映射和错误响应不变。
3. 比对 OpenAPI 或消息 schema 的前后差异，任何变更都需解释。
4. 检查脚本、SDK、合作方和缓存中的字段使用；源码搜索不是完整调用方清单。
5. 发布后按版本观察旧字段使用，且不得记录敏感业务负载。

15. 案例八：纯运费规则 - 取消没有收益的 DI 仪式

15.1 改前

CSHARP

```
public interface IShippingFeeCalculator
{
    decimal Calculate(decimal weight, decimal unitRate);
}

public sealed class ShippingFeeCalculator
    : IShippingFeeCalculator
{
    public decimal Calculate(decimal weight, decimal unitRate)
    {
        return weight * unitRate;
    }
}

public interface IShippingFeeCalculatorFactory
{
    IShippingFeeCalculator Create();
}
```

本文综合建议 - 气味：一个无状态、无副作用、只有一个稳定公式的纯计算，被接口、实现、工厂和 DI 注册包围。抽象没有隐藏易变细节，没有修正依赖方向，测试也不需要替身。导航和接线成本反而超过公式本身。

15.2 推理

Microsoft 的 Clean Architecture 指导支持依赖反转用于核心与基础设施解耦，但并不要求每个本地函数都通过接口和容器调用。[S3]

本文综合建议：先问未来变化是什么。如果只有同一政策内的最低费用和重量边界，明确的值对象与纯函数已经足够；如果未来出现按渠道、区域和合同独立变化的多套政策，再引入命名策略也不迟。

15.3 改后

CSHARP

```
public static class ShippingFeePolicy
{
    public static decimal Calculate(
        decimal weight,
        decimal unitRate,
        decimal minimumFee)
    {
        ArgumentOutOfRangeException.ThrowIfNegative(weight);
        ArgumentOutOfRangeException.ThrowIfNegative(unitRate);
        ArgumentOutOfRangeException.ThrowIfNegative(minimumFee);

        var variableFee = weight * unitRate;
        return decimal.Max(variableFee, minimumFee);
    }
}
```

本文综合建议 - 改进理由：删除的是无收益间接层，不是未来变化能力。函数签名暴露全部输入，没有隐藏 I/O，可直接表格测试。这里保留一个命名良好的政策类，而不是把公式散落在调用者中；“不使用 DI”不等于“不建立业务词汇”。

若运费取决于远程价目表、租户配置或时间，则应把取数副作用放在边界外，先取得明确输入再计算；若政策真正出现独立实现，再以业务名称建策略。这是本文综合建议。

15.4 安全证据

本文综合建议 - 证明包：

1. 搜索所有接口、工厂和 DI 注册调用，确认没有反射或配置解析。
2. 用改前实现结果建立表格测试，覆盖 0、最低费交界和非法负数。

3. 编译所有消费项目，运行启动测试以发现遗留容器注册。
4. 比较异常类型与边界行为；若新增负数拒绝改变旧行为，应拆成单独需求。
5. 删除注册后运行应用启动与关键调用场景，而不只运行纯函数单元测试。

16. 从案例提炼出的共同模式

本文综合建议：八个案例看似分散，实际共享一个判断序列：

1. 先说业务不变项。舍入、库存非负、幂等、错误聚合、报表粒度、字段名。
2. 再找变化轴。政策、存储、供应商、业务拥有者、协议或调度配置。
3. 让依赖指向稳定语义。外部细节适配端口，而非领域反向依赖 SDK。
4. 保留诚实差异。不用开关万能函数强行合并不同政策。
5. 按失败机制选择测试。纯函数、数据库并发、HTTP 契约各需不同证据。
6. 删除发生在证明之后。零引用、低调用、指标下降都不是删除授权。
7. 允许代码变长。显式类型、测试、适配器和错误处理可能增加 LOC，但减少了风险与推理距离。

这套序列就是从“代码瘦身”走向“代码审美”的核心：审美不是看到少，而是看见哪些信息应该存在、应该在哪里存在，以及如何证明它们仍然正确。

第三部分 把审美带进代码评审

17. 按风险顺序阅读，而不是按个人偏好挑刺

一次有效评审不是从命名开始，也不是先数接口。若行为和事务已经错误，讨论 换行风格只会制造虚假的精确感。Google 的工程实践把小变更理解为聚焦的、自包含的概念变更，并建议重构与功能或缺陷变更通常分开。[S13] 这为评审先 确认变更意图与范围提供了工程依据，但不规定下面的具体问题顺序。[S13]

本文综合建议：按以下顺序提问；前一层没有答案时，先不要陷入后一层。

17.1 第一问：这次到底要保持和改变什么

1. 变更类型是纯重构、缺陷修复、功能修改，还是混合批次？
2. 一句话目标是什么？是否包含两个可独立交付的概念？
3. 外部行为、不变项和已知缺陷分别是什么？
4. 哪些行为明确不在本批次改变？

本文综合建议：若作者无法区分“有意变化”和“意外差异”，评审者应先 要求补充目标与基线，而不是猜测代码是否等价。

17.2 第二问：高损失边界在哪里

1. 是否触及金额、权限、库存、并发、事务或数据迁移？
2. 是否改变 API、JSON、消息、数据库对象或公开类型？
3. 是否改变外部副作用的次数、顺序、重试或幂等键？
4. 是否有低频调度、反射、DI 扫描、配置或脚本入口？

.NET 裁剪分析文档说明静态可达性对反射和运行时装载存在限制。[S7][S8] 因此，“搜索不到引用”在第二问中只能是候选证据，不能结束调查。

17.3 第三问：责任和依赖是否更诚实

1. 改动隐藏了哪项可能变化的设计决策？
2. 新边界是否有明确拥有者与业务名称？
3. 核心策略是否仍引用 ORM、HTTP SDK 或供应商 DTO？
4. 新接口是否隔离副作用或修正依赖方向，还是只做逐方法转发？
5. 理解修改需要跨越的文件、配置与运行组件是更多还是更少？

Parnas 的研究支持围绕应隐藏的设计决策进行分解，而不是只按处理步骤拆分。[S2] Microsoft 的 .NET 架构指导支持让基础设施依赖应用核心的抽象。[S3] 具体模块名和项目布局仍是本文综合建议，不能由两项来源机械推出。

17.4 第四问：重复真的属于同一知识吗

1. 是相同文字，还是相同行为、业务知识与变化方向？
2. 两处是否由同一政策和同一业务所有者驱动？
3. 合并后会不会出现 bool 开关、字符串模式或委托拼装？
4. 新抽象能否用一个准确的业务名表达？

5. 保留少量重复是否反而能保护模块边界？

CPD 能发现复制粘贴块，却不判断业务语义是否等价。[S11] 所以评审意见不能只写“重复三次，请抽象”；它应指出共同知识与同步变化证据，或承认这只是待调查候选。

17.5 第五问：证据覆盖了真实失败机制吗

1. 纯计算是否有边界表格和失败条件测试？
2. SQL、映射、事务是否在兼容数据库上验证？
3. 并发是否以竞争请求和数据约束验证，而非只做单线程单测？
4. HTTP 或消息是否覆盖超时、拒绝、重试和重复响应？
5. 契约是否比较字段、空值、顺序、错误码与旧样例？
6. 测试自身是否稳定，断言是否真正区分正确与错误？

Microsoft 的测试指导区分测试层次，并把关键业务逻辑与可失败条件置于单纯覆盖率之前。[S6] 其 Well-Architected 指导还要求关注测试自身质量和高价值、稳定的回归测试。[S9]

17.6 第六问：失败后能否定位和恢复

1. 批次能否独立回退？数据和外部副作用是否允许回退？
2. 新旧路径如何区分，差异如何计数？
3. 是否保留关联 ID、失败分类与部署版本？
4. 哪项信号触发暂停、回滚或前向修复？
5. 剩余未测场景是否被明示，而不是被“测试通过”覆盖？

Fowler 的重构指导支持小步、保持行为的转换。[S12] 将回滚、观测和停止条件写入评审，是本文综合建议。

17.7 最后才问表达细节

本文综合建议：在前六问成立后，再审查名称、方法长度、控制流、注释、异常信息与格式。表达仍然重要，因为意图可见影响未来修改；但表达意见应能指向理解成本或错误风险，而不是“我喜欢另一种写法”。

18. 审美红旗：看到后应停下来问证据

本文综合建议：下列信号不是自动缺陷，而是需要追问的红旗。

- “只是重构”，但接口快照、SQL 结果或副作用次数发生变化。
- LOC 明显下降，却同时删除测试、类型、错误处理或解释原因的注释。
- 一个函数出现多个 bool、模式字符串或 object 参数来容纳差异。
- 每个实现都有同名接口，却没有隔离变化、副作用或引用方向。
- 领域模型直接暴露 ORM 实体、HTTP SDK 或供应商枚举。
- 新模块按 Controller/Service/Repository 分层，却让同一规则复制三次。
- “未使用”结论只来自 IDE，未检查反射、DI、配置、序列化和调度。
- 单元测试全部通过，却修改真实数据库事务、锁或排序语义。
- 覆盖率提高，但新增断言只验证“不抛异常”。
- catch (Exception) 后记录一行并返回成功或空集合。

- 新抽象难以命名，只能叫 Manager、Helper、Common 或 GenericService。
- 重构和需求修复混在一个大提交，无法判断差异来自哪一类改变。
- 删除兼容字段，却没有调用方清单、退场窗口和明确拒绝行为。
- 指标报告没有工具版本、排除范围，或把测试与生产代码混在一起。
- 代码更短，但理解一次修改需要跨更多文件、配置和运行组件。

红旗的正确输出不是立即否决，而是一个可回答的问题。例如：

TEXT

这里删除了零引用 Job。请补充任务表、配置和程序集扫描的核查证据，并说明观测窗是否覆盖该 Job 的月度周期；否则建议先标 UNCONFIRMED。

这比“我觉得不安全”更具体，也比“静态分析没问题”更诚实。以上评审表达是 本文综合建议。

19. 写出能推动改进的评审意见

本文综合建议：一条高质量意见包含四部分：观察、风险、请求、通过条件。

TEXT

观察：当前先读取 AvailableQuantity，再单独写回。
风险：两个并发请求可能基于同一旧值都返回成功。
请求：请把竞争判定下沉为数据库条件原子更新，并将保留记录放入同一事务。
通过：并发集成测试证明成功总量不超过初始库存，失败注入时两项写入均回滚。

意见应区分严重级别：

- 阻断：可能改变契约、数据、权限、事务、并发或不可恢复副作用。
- 需要修改：明显增加耦合、隐藏不变项或缺少与风险匹配的证据。
- 建议：可读性、命名或局部简化，有合理替代即可。
- 问题：评审者缺少上下文，请作者补事实，不预设答案。

不要把个人风格升级成阻断，也不要将高风险问题降格为“nit”。这套严重级别 是本文综合建议。

可打印的一页代码审美评审卡

用途：打印或复制到评审描述中。所有条目均为本文综合建议。

A. 先定性

- ☐ 我能用一句话说出本批次的唯一目标。
- ☐ 已区分纯重构、功能变化、缺陷修复与已知缺陷。
- ☐ 已列出接口、业务、数据、事务、并发和副作用不变项。

B. 再找风险

- ☐ 已检查金额、权限、库存、迁移、并发和外部幂等。
- ☐ 已检查 API、JSON、消息、数据库和公开类型契约。
- ☐ 删除候选已检查反射、DI、配置、序列化、插件、调度与脚本。

C. 看边界

- ☐ 变化有明确拥有者，内部表示没有泄漏。
- ☐ 稳定策略不依赖 ORM、HTTP SDK 或供应商 DTO。
- ☐ 新接口隔离了真实变化或副作用，不是空转发。
- ☐ 重要禁止依赖可由项目引用、分析器或架构测试检查。

D. 判重复

- ☐ 已区分相同文字、相同行为、相同知识和相同变化方向。
- ☐ 合并不会引入 bool 开关、字符串模式或跨模块故障面。
- ☐ 若保留重复，已说明不同拥有者或不同变化原因。

E. 看证据

- ☐ 测试层与失败机制匹配，关键断言能区分正确和错误。
- ☐ 数据库、并发和外部契约没有只靠纯单元测试证明。
- ☐ 比较了空值、0、排序、错误、行数、舍入和副作用次数。
- ☐ 指标只用于排序与趋势，没有替代行为证据。

F. 准备恢复

- ☐ 批次小而聚焦，能独立验证和回退。
- ☐ 数据或外部副作用不可回滚时已有前向修复方案。
- ☐ 有关联 ID、失败分类、版本标识和明确停止条件。
- ☐ 剩余风险、未测场景与人工确认项已经写明。

一句话结论

TEXT

结论：PASS / PAUSE / ROLLBACK / NEEDS_EVIDENCE

不变项：

关键证据：

动态入口核查：

剩余风险：

下一步：

第四部分 练习：把概念变成判断

练习说明

本文综合建议：先独立作答，再看后面的解析。每题写下“事实、推断、证据 缺口”三栏；不要只给一个重构后的代码片段。练习答案不唯一，评分重点是能否解释边界、风险和证明方式。

练习 1：诊断一个短函数

阅读代码，找出至少六个影响审美的事实或未知项。按“意图、责任、不变项、可测试、可观察”分类，并写出你在重构前需要确认的三个业务问题。

CSHARP

```
public decimal Calc(List<Row> rows, bool old)
{
    return rows.Where(x => x.S == 1)
        .Sum(x => Math.Round(x.Q * x.P * (old ? 1m : 1.13m), 2));
}
```

练习 2：比较两个通知设计

设计 A：领域服务直接注入 `HttpClient`，构造供应商 JSON。设计 B：领域服务依赖 `IShipmentNoticePort`，HTTP 适配器映射供应商协议。

回答：

1. 哪个设计的源码依赖方向更稳定，为什么？
2. 哪些情况下设计 A 仍可接受？
3. 设计 B 的端口若返回 `bool`，还隐藏了什么重要信息？
4. 写出三个必须由契约测试而非纯领域单元测试覆盖的行为。

练习 3：重构草图 - 拆开导入编排

现有 `ImportService.Import(file, bool isRetail, bool legacy, bool stopEarly)` 同时负责读 Excel、解析日期、查客户、校验、写数据库和生成错误文件。

画出一个不超过六个组件的重构草图，标出：

- 哪些是纯规则；
- 哪些是外部副作用端口；
- 哪些重复应先保留；
- 第一个可逆批次是什么；
- 你会如何证明第一批没有改变导入结果。

练习 4：边界设计 - 报表查询

业务说：“列出日期范围内所有客户的净额，没有发票也要显示；金额为空时按 0；按客户编码排序，最后一行是总计。”

请设计查询对象、行类型和读取端口的 C# 签名。说明日期的包含边界、明细行与总计行如何区分，以及为什么不能只返回 `List<dynamic>`。

练习 5：死代码证据

`LegacySettlementJob` 在解决方案中零引用，最近 60 天日志无调用。它曾用于 季度结算。请给出：

1. 当前能得出的最强结论；
2. 还要查的至少六类入口或证据；
3. 何时可进入 `DEPRECATION_WINDOW`；
4. 删除后的明确失败行为和回退方案。

练习 6：库存并发

评审者看到 `if (stock >= quantity) stock -= quantity;` 已被提取成一个纯函数，单元测试覆盖率 100%，于是批准重构。指出这个结论为什么不足。给出一个最小 并发集成测试的输入、并发动作和四条断言。

练习 7：是否去重

合同模块与促销模块都有：

```
CSHARP
```

```
return amount * 0.9m;
```

合同折扣由长期协议决定，促销折扣由每周活动决定。请在以下方案中选择并说明：

1. 立即提取 `CommonDiscount.Calculate`；
2. 建立带 `isPromotion` 的通用函数；
3. 暂时分别保留，并给各自准确命名；
4. 其他方案。

你的回答必须包含“未来什么证据会改变当前决定”。

练习 8：改写评审意见

把下面三句话改成包含观察、风险、请求和通过条件的工程意见：

1. “这个写法不优雅。”
2. “三次重复必须抽象。”
3. “测试绿了，应该没问题。”

练习 9：指标判断

某次瘦身结果如下：生产 LOC -18%，测试 LOC -30%，重复率 -40%，覆盖率从 82% 降到 68%，两个复杂方法合并成一个 9 个 bool 参数的方法。构建与已有 测试通过。

使用本文记分卡写出结论：哪些数字是信号，哪些是红旗，还缺哪些行为证据？能否判为成功？为什么？

练习 10：设计一个可逆批次

现有发货服务同时保存状态、调用通知供应商、重试三次并吞掉最终异常。目标是 建立通知端口，但本批次不得改变发货与失败行为。

请写出两个按顺序执行的小批次，每批说明：范围、不变项、验证和回退。特别解释为什么“顺便改成 Outbox”不应混进第一个纯结构批次。

练习参考答案与解析

以下是推理示范，不是唯一答案。判断依赖项目事实；若你的方案能清楚表达 不变项、依赖方向、风险和证据，也可能同样成立。

练习 1 解析

验收要点：至少列出六个事实或未知项、三个待确认业务问题，并说明至少一项要先由特征测试固定的不变项。

本文综合建议：可诊断出：`Calc` 不表达金额类型；`S == 1` 是魔法状态；`old` 不说明旧的是什么；税率和舍入顺序隐藏；逐行舍入是否为真实政策未知；没有折扣、负数、空集合与精度约束；纯函数易测但没有任何测试证据；结果没有币种；没有观察异常数据的方式。

重构前可问：状态 1 是否指已审核？旧模式是免税、未税还是历史错误兼容？舍入应逐行还是汇总？进一步还应确认税率是否配置、不同币种精度和负数语义。先把答案写成特征测试，再命名枚举和政策；不要从“提取接口”开始。

练习 2 解析

验收要点：必须比较依赖方向，给出设计 A 的条件性例外，指出 `bool` 的信息损失，并写出三个具体的适配器契约断言。

本文综合建议：B 通常让基础设施实现依赖核心端口，供应商变化停在适配器，符合 .NET 架构指导所描述的依赖反转方向。[S3] A 在一次性、边界清楚、没有领域层且协议就是应用唯一责任的小工具中可能足够；不能因示例规模小就绝对化。

`bool` 只表达二分成功，可能丢失永久拒绝、可重试失败、已接受异步处理和重复请求等状态。契约测试至少覆盖精确字段/URL、非成功状态分类、超时与幂等键；这些属于 HTTP 适配器行为，不应由纯领域测试伪造为已证明。

练习 3 解析

验收要点：草图不超过六个组件，清楚区分纯规则与 I/O，第一批只有一个变化轴，并包含可比较的输入输出证据与回退单位。

本文综合建议：一种六组件草图是：`ImportUseCase` 编排；`IFileRowReader` 读文件；两个独立的 `RetailRowValidator` 与 `ContractRowValidator`；`IImportRepository` 写数据；`IErrorReportWriter` 生成错误文件。无业务含义的日期解析可为共享纯函数，两个校验器的业务分支先保留。

第一批只把现有文件读取包进 `IFileRowReader`，默认实现仍调用旧代码；不改校验、写入或错误次序。用同一脱敏文件对比行数、行号、解析值、错误列表与输出文件哈希；构建和导入功能测试通过后独立提交。第二批再包围写入或提取纯校验。每批只有一个变化轴，因此失败时能定位和回退。

练习 4 解析

验收要点：签名必须表达日期边界、行种类、金额空值语义和稳定排序，并解释为何 `dynamic` 不能承担报表契约。

CSHARP

```
public sealed record CustomerNetQuery(
    DateOnly FromInclusive,
    DateOnly ToExclusive);

public enum ReportRowKind
{
    Detail,
    Total
}

public sealed record CustomerNetRow(
    ReportRowKind Kind,
    string? CustomerCode,
    decimal Amount,
    int SortOrder);

public interface ICustomerNetReportReader
{
    Task<IReadOnlyList<CustomerNetRow>> QueryAsync(
        CustomerNetQuery query,
        CancellationToken cancellationToken);
}
```

本文综合建议：**ToExclusive** 消除结束日歧义；**Kind** 避免把“合计”伪装成客户编码；**SortOrder** 或明确 SQL 排序保证总计尾行；**decimal** 与非空金额表达 **NULL** 已在查询边界归零。**dynamic** 无法在编译期固定字段、空值和粒度，也让契约漂移更难发现。安全证据应比较客户全集、0/NULL、排序和尾行。

练习 5 解析

验收要点：结论必须是 **UNCONFIRMED**，至少六类动态或运行证据齐全，并给出进入退场、删除后的失败方式和可操作回退路径。

本文综合建议：当前最强结论是 **UNCONFIRMED**：无显式引用且 60 天观测窗无调用，但季度周期未覆盖。还应查反射、DI 扫描、各环境配置、任务数据库、部署脚本、运维手册、插件目录、公开调用方、审计日志和负责人。

只有业务确认替代路径、所有环境完成迁移、观测覆盖至少一个相关周期、启动与触发测试通过并准备回退时，才进入退场窗口。删除后遇到旧任务键应明确报 **Unknown job handler** 并报警，不应静默跳过；回退可恢复旧程序集与任务配置，同时评估删除期间是否漏跑及如何补偿。

练习 6 解析

验收要点：必须否定“100% 覆盖率足够”的推断，给出竞争输入、并发动作、四条数据断言及数据库环境要求。

本文综合建议：纯函数只证明“给定一个快照如何计算”，不保证多个请求获得互斥快照或原子写回。100% 覆盖率也不描述数据库隔离、条件更新和约束。

最小测试可准备库存 10，并发发起 20 个数量 1 的不同幂等请求。断言：恰有 10 个成功；最终库存为 0 且不为负；成功请求各有且仅有一条保留记录；失败请求无记录。再用相同幂等键并发重试，断言不重复扣减。测试必须在兼容数据库上运行并记录隔离级别。

练习 7 解析

验收要点：必须选择或提出一个方案，说明变化轴和故障面，并写出会触发重新评估的可观察证据。

本文综合建议：当前优先选 3。两段文字和当前计算相同，但业务知识与变化方向不同；通用函数或 bool 会制造跨模块依赖。应分别命名 **ContractDiscount** 与 **PromotionDiscount**，各自测试并可短注释“独立政策”。

未来若组织确认两者受同一费率表、同一所有者和同一变更流程驱动，历史修改 长期同步，且共同抽象能准确命名、不会扩大故障面，再重新评估共享。这里保留 的是边界，不是拒绝复用。

练习 8 解析

验收要点：三条意见各自包含观察、风险、请求和通过条件，且严重性与 实际风险相称，不能把个人偏好包装为阻断。

本文综合建议：可改写为：

1. “观察：此方法名为 **Process**，同时更新状态并发送 HTTP。风险：调用者 无法从签名看出副作用，失败边界也不清。请求：请用业务名称分开状态策略 与通知端口。通过：测试分别覆盖状态保存和通知拒绝时的既定行为。”
2. “观察：三段令牌相似，但分别属于合同、促销和会员政策。风险：立即合并 可能把不同变化轴耦合。请求：请说明共同业务知识与所有者；若不存在，保留 并分别命名。通过：决策记录说明合并或保留依据。”
3. “观察：现有单测通过，但本批修改了条件更新 SQL。风险：单线程替身未覆盖 数据库竞争。请求：增加兼容数据库并发测试。通过：成功总量受库存约束、数据不为负、失败事务无孤儿记录。”

练习 9 解析

验收要点：必须区分趋势指标与行为证据，识别测试下降和 bool 万能函数，并给出暂停而非通过的可复核理由。

本文综合建议：生产 LOC 和重复率下降只是候选收益；删除 30% 测试、覆盖 率显著下降、9 个 bool 的万能函数是强红旗。复杂方法数量下降可能只是把分支 集中到更危险的地方。构建与现有测试通过不能说明被删除的断言或业务组合仍 受保护。

缺少：关键不变项清单、删除测试理由、各 bool 合法组合、契约与数据对比、复杂度前后值、依赖方向、回滚与可读性审查。按本文记分卡，行为保持与风险 降低无法达到门槛，因此不能判成功；应暂停、恢复关键测试，拆开不同政策并 重新验证。

练习 10 解析

验收要点：两个批次都要写出范围、不变项、验证和回退，并明确说明 Outbox 改变了哪些行为边界，为什么必须另立批次。

本文综合建议：第一批只引入 **IShipmentNoticePort** 与旧供应商适配器，让现有调用通过端口，保持三次重试、异常吞掉和状态顺序不变。特征测试记录 调用次数、请求体、异常后的返回和日志；适配器可一键改回直接调用。

第二批把重试政策显式封装，但仍保留三次和最终失败行为；测试用可控适配器 验证重试次数、取消和既有日志。每批独立提交。Outbox 会改变事务、最终一致性、重试主体和运维模型，是功能与架构行为改变，应单独设计、评审和迁移，不能 伪装为通知端口提取的一部分。

第五部分 自测：能否把审美变成可验证判断

20. 十题自测

作答方式：先遮住答案，用自己的话作答。每题 0、1、2 分；不确定时写出 需要补的证据，不把猜测当作事实。本节不代表读者已经完成任何训练。

1. 为什么“生产 LOC 下降 20%”不能单独证明一次瘦身成功？请列出两个必须 同时检查的维度。
2. Parnas 式的信息隐藏主要围绕什么建立模块边界？它与按 Controller、Service、Repository 拆目录有什么根本区别？
3. 一个核心服务改为依赖 `IPaymentGateway` 后，如何判断这是真正的端口，还是 对 SDK 的逐方法转发？写出两项检查。
4. 两段相同的折扣公式何时应暂时保留？请给出业务知识或变化方向上的理由。
5. IDE 显示一个 Job 没有引用。删除前至少要检查哪四类非静态入口？
6. 为什么库存的“先读数量、再减数量”即使拥有 100% 单元测试覆盖也不安全？写出数据库级解决方向。
7. 报表要求“无记录显示 0，合计永远在最后”。最少应在结果契约中明确哪三项？
8. 纯重构批次的测试已经通过，但 JSON 字段从 `customerCode` 变成 `customerId`。这应视为哪类问题？下一步应补什么证据？
9. 把三段相似验证合为一个含五个 bool 的函数，通常暴露什么风险？何时才可能 接受这种设计？
10. 请把“这里不优雅”改写成一句可执行的评审意见，必须包含观察、风险、请求 和通过条件。

自测答案、评分与补救路径

21. 参考答案

1. LOC 只描述体量趋势，不能证明行为、契约或风险改善。还应检查关键不变项的 测试/验收，以及依赖、复杂度或可回退性等风险证据。[S1][S5]
2. 边界围绕可能独立变化、应被隐藏的设计决策建立；目录分层只是在放置文件，若同一规则仍横跨各层，就没有实现信息隐藏。[S2]
3. 检查端口是否使用稳定业务语言、是否隔离网络/SDK 副作用；再检查核心是否不再引用具体 SDK，替换实现时调用者是否无需了解供应商 DTO。[S3]
4. 当两段公式由不同政策、所有者、发布节奏或错误语义驱动时，文字相同不等于同一知识；先保留并准确命名，直到有共同变化的证据。[S11]
5. 至少检查反射或程序集扫描、DI 注册、配置或任务表、调度器、序列化、插件与运维脚本。静态可达性不能完全覆盖动态行为。[S7][S8]
6. 覆盖率不产生互斥或原子性。应采用数据库条件更新，例如仅在可用量足够时更新一行，并在同一事务内写保留记录，以约束和幂等键处理重试。
7. 明确集合粒度与是否包含零记录客户、金额的 NULL 到 0 语义、明细和合计的行类型与稳定排序/尾行规则。
8. 这是外部 JSON 契约改变，不是纯重构。先比较 OpenAPI 或消息 schema，加入旧样例序列化/反序列化契约测试，并核查消费者和兼容窗口。
9. 多个 bool 常把独立变化轴压进一处，产生非法组合、分支爆炸与测试组合膨胀。只有每个选项独立、局部、含义清楚且组合确有业务意义时，才可能接受。
10. 例如：“观察：方法同时更新发货状态并直接发送 HTTP。风险：通知失败边界与重复发送语义不可见。请求：请将通知协议放到端口后，并保留既有行为。通过：契约测试验证请求字段，特征测试验证失败和重试次数。”

22. 评分与补救

每题按以下尺度评分：

- 2 分：结论正确，能说出风险或边界，并给出匹配的证据或通过条件。
- 1 分：抓住部分概念，但没有说明为何或缺少实际验证方式。
- 0 分：只给口号、只谈风格，或把工具信号误当成安全证明。

总分	当前解释	下一步练习
17-20	已能把多数审美判断连接到边界与证据	选一个低风险模块，完成一次评审卡演练
13-16	概念已出现，但风险和验证仍可能脱节	重做答错题，并为每题补一条可运行或可核查证据
8-12	易被“更短”“测试绿了”或“有接口”误导	先做第 1、5、6、7 题对应案例，再阅读评审卡
0-7	还没有稳定的判断框架	从第一部分和案例二、五、六开始，逐段写事实/推断/证据缺口

分数只是定位学习缺口，不是能力认证，也不代表任何工程变更已经被验证。

第六部分 反思与 14 天能力微练习

23. 一次真实评审后的反思模板

在一次已发生的评审、重构或排障后填写。不要把“已完成”预先写入模板；每项 只记录实际观察到的事实，并分开记录推断和待核查项。

TEXT

日期：

对象（模块/变更/场景）：

唯一目标：

我观察到的事实：

1.

2.

我做出的推断，以及依据：

1.

2.

最重要的不变项（接口/数据/事务/并发/副作用）：

1.

2.

我看到的变化轴与边界：

1.

2.

证据已具备：

证据仍缺失：

动态入口或低频场景：

可回退点或前向修复方案：

我写出的评审意见：

收到的反馈或运行结果：

下一次我要先问的问题：

24. 14 天微练习路径

每次建议 20-30 分钟。选择脱敏、低风险、可只读检查的材料；未获得授权时，不要删除代码、修改生产配置或运行写入性操作。完成证据由读者自己记录，本表 不宣称任何日期已经完成。

天数	微练习	产出与自检
1	选一个 30 行以内的方法，标注意图、输入、输出和隐藏副作用	写出三条事实与一条未知项
2	为同一方法列不变项	至少覆盖错误、空值和一个业务边界
3	画出它的直接依赖和调用者	区分源码依赖、数据依赖和运行依赖
4	找两处相似代码	写清文字、知识和变化方向是否相同
5	选择一个接口	判断它隔离的是副作用、替换性还是纯仪式
6	找一个“零引用”候选	建立 E1-E3 证据清单，不删除任何内容
7	阅读一条 SQL 或报表查询	记录粒度、集合、空值语义与排序
8	为一个金额或日期函数列边界表	写输入、期望输出和舍入/时区假设
9	对一个写操作做并发思想实验	写竞争步骤、数据库不变项和失败注入点
10	选择一个 HTTP 或消息调用	记录字段、超时、拒绝、重试和幂等键
11	使用评审卡审阅一个小提交	至少写一条观察-风险-请求-通过意见
12	设计一个只改变结构的小批次	写范围、不变项、验证、回退和停止条件

天数	微练习	产出与自检
13	回看一个曾经的重构	区分当时的事实、猜测、已验证结果和剩余风险
14	完成一次反思模板与十题自测	把最低分项映射为下一个两周的练习主题

若练习触及数据库、外部通知、权限、账务、库存或生产数据，先把练习改为 只读分析、测试替身或经授权的受控环境；安全边界比练习速度重要。

参考文献与注释

以下 S1-S14 与本文引用一一对应，按来源的公开范围、版本和链接边界标注。“支持”只说明本文可据以转述的范围；“限制”说明不应从该来源推出的结论。访问日期均为 2026-07-29。

- [S1] 规范性标准（公开元数据；全文元数据用途）。International Organization for Standardization (ISO) / IEC，[ISO/IEC 25010:2023 - Systems and software engineering - Systems and software Quality Requirements and Evaluation \(SQuaRE\) - Product quality model](#)，第 2 版，2023-11（发布 2023-11-15）。支持：该质量模型面向 ICT/软件产品，可用于质量需求、评价、测试目标、验收与度量。限制：公开页不是免费标准全文；本文不据此声称具体子特性定义、计量方法或强制阈值。访问：2026-07-29。
- [S2] 原始研究。D. L. Parnas，Association for Computing Machinery，[On the Criteria To Be Used in Decomposing Systems into Modules](#)，*Communications of the ACM* 15(12)，1053-1058，1972-12-01。支持：比较模块分解方式及其对灵活性、可理解性和开发时间的影响，是信息隐藏导向分解的一手来源。限制：论文讨论的是原则与示例，不提供现代系统的自动分层算法或团队流程。访问：2026-07-29。
- [S3] 工程指导。Microsoft，[Common web application architectures](#)，.NET Architecture 文档，页面最后更新 2023-03-07。支持：依赖反转与 Clean Architecture 中由基础设施依赖应用核心的方向。限制：参考架构是指导，不要求所有应用使用 Clean Architecture。访问：2026-07-29。
- [S4] 工具专用执行。Microsoft，[CA1506: Avoid excessive class coupling \(code analysis\)](#)，页面最后更新 2023-11-14。支持：唯一类型引用计数、可配置阈值、低耦合/高内聚的维护性建议。限制：适用于 C# / Visual Basic；计数和阈值不能单独决定重构是否正确。访问：2026-07-29。
- [S5] 工具专用执行。Microsoft，[CA1505: Avoid unmaintainable code \(code analysis\)](#)，页面最后更新 2023-04-22。支持：可维护性指数包含 LOC、程序体积和圈复杂度，阈值可配置且可在有理由时抑制。限制：仅是 .NET 分析器规则，不是跨语言质量标准或删除代码授权。访问：2026-07-29。
- [S6] 工程指导。Microsoft，[Test ASP.NET Core MVC apps](#)，.NET Architecture 文档，页面最后更新 2022-09-21。支持：测试层次、条件/错误路径与关键业务逻辑优先于覆盖率数字。限制：以 ASP.NET Core 应用说明，测试比例和类型必须按系统风险调整。访问：2026-07-29。
- [S7] 工程指导。Microsoft，[Understanding trim analysis](#)，页面最后更新 2025-11-19。支持：静态可达性分析的入口与限制；反射、动态装载等运行时行为为何不能由静态分析完全判定。限制：讨论 .NET 裁剪，不是任意语言死代码分析的完备理论。访问：2026-07-29。
- [S8] 工程指导。Microsoft，[Fixing trim warnings](#)，页面最后更新 2025-11-19。支持：裁剪警告意味着潜在行为改变/崩溃，存在警告时应在裁剪后充分测试；动态模式和抑制的风险。限制：针对 .NET 发布裁剪，需结合项目实际运行时入口。访问：2026-07-29。
- [S9] 工程指导。Microsoft，[Architecture strategies for testing](#)，Azure Well-Architected Framework，页面最后更新 2026-03-31。支持：验证测试本身、回归集优先放最有价值且稳定的测试，并分层自动运行。限制：云工作负载工程指导，不是所有离线/嵌入式系统的完整测试策略。访问：2026-07-29。

- [S10] 工具专用执行 (提交固定)。TNG Technology Consulting GmbH, [ArchUnit README](#), 标签 [v1.4.2](#) 对应提交 [e45aaa20543b74daeea09f501b8890742dc7dd8d](#), 发布 2026-04-18。支持：以普通 Java 单元测试检查包、类、层、切片及循环依赖的架构规则。限制：Java 字节码工具；仅作为“边界可执行化”的示例。访问：2026-07-29。
- [S11] 工具专用执行。PMD 项目, [Finding duplicated code with CPD](#), PMD 用户文档, 页面滚动发布 (可访问版本于 2026-07-29 获取)。支持：CPD 是复制粘贴检测器, 可定位大型项目中的重复块。限制：检测结果不证明语义等价、也不提供安全删除/合并结论。访问：2026-07-29。
- [S12] 原作者工程指导。Martin Fowler, [Refactoring](#), 作者维护的重构入口与定义页。支持：以小的、保持行为的转换逐步重构, 降低错误与长时间破坏系统的风险。限制：作者的工程方法论, 不是正式标准；仍需项目自身的行为基线。访问：2026-07-29。
- [S13] 平台工程指导。Google, [Small CLs](#), Google Engineering Practices。支持：一个 CL 聚焦一个自包含变更；规模是概念聚焦而非简单行数函数；重构与功能/缺陷变更通常分开, 相关测试与纯重构同行。限制：Google 的评审实践, 不是跨组织标准；具体批次大小由项目风险和评审者判断。访问：2026-07-29。
- [S14] 规范性标准 (公开元数据；正文未取用)。International Organization for Standardization (ISO) / IEC, [ISO/IEC 5055:2021 - Information technology - Software measurement - Software quality measurement - Automated source code quality measures](#), 第 1 版, 2021-03。支持：公开摘要说明自动化源代码质量度量以检测和计数架构/编码实践违规为基础, 并关联运行风险或过高成本；公开页在访问日显示阶段 90.60 (Under review)。限制：公开落地页不展示 235 页正文, 本文不声称其具体算法、违规集合或阈值。访问：2026-07-29。